

Shurokkha — Shelters, Warehouses, Donations

প্রেজেন্টেশন গাইড

এই গাইডটি আপনার কালকের ডিফেন্সের জন্য। স্যার যেকোনো লাইন, যেকোনো প্রশ্ন করুক — উত্তর এখানে রেডি আছে। **নিয়ম:** টেকনিক্যাল টার্ম, SQL কিওয়ার্ড, ফাংশনের নাম ইংরেজিতে; বাকি সব সহজ বাংলায়।

সূচিপত্র

- এক নজরে সবকিছু (কুইক ম্যাপ)
- র‍্যান্ডম কোড লাইন ব্যাখ্যা — Controller, Model, Frontend
- ডাটাবেজ JOIN ও লজিক (ট্র্যাপ এলাকা)
- কলাম, গারবেজ ডাটা ও অমিল (Discrepancy)
- র‍্যাপিড-ফায়ার প্রশ্নোত্তর
- সব টেবিল ও কলামের রেফারেন্স টেবিল

১. এক নজরে সবকিছু

আপনি তিনটি মডিউল বানিয়েছেন। প্রতিটির তিনটি স্তর (3-tier):

স্তর	কাজ	ফাইলের ধরন
ডাটাবেজ	ডাটা সংরক্ষণের নিয়ম	database/migrations/08,09,10
ব্যাকএন্ড (Laravel)	ডাটা আনা/পাঠানো/মুছা	Controller + Model + routes/api.php
ফ্রন্টএন্ড (React/Next.js)	ইউজারকে দেখানো ও ফর্ম	operations-workspace.tsx

তিনটি মডিউল:

মডিউল	টেবিল	কী ধারণ করে
 Shelter	shelters	আশ্রয়কেন্দ্র (নাম, ধারণক্ষমতা, কতজন আছে)
 Warehouse	warehouses	ত্রাণ গুদাম (নাম, লোকেশন, ম্যানেজার)
 Donation	donations	দান (কী দান হলো, পরিমাণ, স্ট্যাটাস)

👤 **এক লাইনে বলুন:** "স্যার, আমি ত্রাণ ব্যবস্থাপনার তিনটি অংশ — আশ্রয়কেন্দ্র, গুদাম ও দান — পুরো স্ট্যাকে (ডাটাবেজ থেকে শুরু করে ইউজার ইন্টারফেস পর্যন্ত) তৈরি করেছি।"

২. র‍্যান্ডম কোড লাইন ব্যাখ্যা

স্যার যেকোনো লাইনে আঙুল রাখতে পারেন। নিচে প্রতিটি ধরনের লাইনের "মানে কী" দেওয়া আছে।

২.১ — Controller ফাইল (যেমন `ShelterController.php`)

তিনটি কন্ট্রোলারই (Shelter, Warehouse, Donation) একই প্যাটার্ন মানে। একটা বুঝলে তিনটাই বোঝা।

```
<?php // লাইন ১: এটা একটি PHP ফাইল
namespace App\Http\Controllers\Api\V1\Admin; // ফাইলটি কোন "ফোল্ডার-পরিচয়ে" আছে
use App\Http\Controllers\Controller; // মূল কন্ট্রোলার ক্লাস যুক্ত করা
use App\Models\Shelter; // Shelter মডেল ব্যবহার হবে
use Illuminate\Http\JsonResponse; // রেসপন্স হবে JSON আকারে
use Illuminate\Http\Request; // HTTP রিকোয়েস্ট ধরার জন্য
```

```
class ShelterController extends Controller // ক্লাস শুরু, মূল কন্ট্রোলারের গুণ পায়
{
    public function index(): JsonResponse // GET রিকোয়েস্ট: সব শেল্টার দেখাও
    {
        $shelters = Shelter::latest()->get(); // সব শেল্টার, নতুন আগে
        return response()->json(['data' => $shelters]); // JSON আকারে ফেরত দাও
    }
}
```

লাইন-ধরে স্যারকে বোঝানোর শব্দ:

- `index()` → "এটি লিস্ট মেথড, সব রেকর্ড পড়ে এনে ফ্রন্টএন্ডে পাঠায়।"
- `Shelter::latest()` → "Eloquent দিয়ে সব শেল্টার আনছি, সর্বশেষ যুক্তটা আগে আসবে।"
- `->get()` → "ক্যোয়ারি চালিয়ে ফলাফল সংগ্রহ করছি।"
- `response()->json(...)` → "ডাটাটা JSON ফরম্যাটে রেসপন্স হিসেবে পাঠাচ্ছি, যা ফ্রন্টএন্ড বুঝতে পারে।"

```
public function store(Request $request): JsonResponse // POST: নতুন শেল্টার বানাও
{
    $validated = $request->validate([ // ইনপুট যাচাই করা হচ্ছে
        'shelter_name' => 'required|string|max:150', // নাম আবশ্যিক, টেক্সট, সর্বোচ্চ ১৫০ অক্ষর
        'capacity' => 'required|integer|min:0', // ধারণক্ষমতা আবশ্যিক, পূর্ণসংখ্যা, ০+
        'occupancy' => 'required|integer|min:0', // কতজন আছে, ০+
        'area_id' => 'nullable|integer', // এরিয়া ঐচ্ছিক (খালি রাখা যায়)
        'status' => 'sometimes|string|max:50', // স্ট্যাটাস থাকলে যাচাই হবে
    ]);

    $shelter = Shelter::create($validated); // যাচাই-পাস ডাটা দিয়ে রেকর্ড বানাও
    return response()->json(['data' => $shelter], 201); // 201 = সফলভাবে তৈরি হয়েছে
}
```

ট্র্যাপ প্রশ্ন ও উত্তর:

- "এক লাইনে `required|integer|min:0` কী?" → "এগুলো ভ্যালিডেশন রুল, পাইপ (|) দিয়ে আলাদা। `required` মানে খালি রাখা যাবে না, `integer` মানে পূর্ণসংখ্যা, `min:0` মানে ঋণাত্মক চলবে না।"
- "201 আর 200 -এর তফাত?" → "200 মানে ঠিক আছে, 201 মানে 'নতুন কিছু তৈরি হয়েছে'। নতুন রেকর্ড বানাতে 201 দেওয়া ভালো প্র্যাকটিস।"
- "`nullable` কেন দিলে `area_id` -এ?" → "কারণ সব শেল্টার শুরুতে কোনো নির্দিষ্ট এলাকার আন্ডারে নাও থাকতে পারে। ডাটাবেজেও আমরা `area_id NULL` রেখেছি।"

```
public function destroy(Shelter $shelter): JsonResponse // DELETE: শেল্টার মুছে
{
    $shelter->delete(); // রেকর্ড মুছে ফেলো
    return response()->json(null, 204); // 204 = মুছে ফেলা হয়েছে, কিছু ফেরত নে
}
```

- "প্যারামিটারে `Shelter $shelter` কীভাবে এলো?" → "এটা **Route Model Binding**। URL-এর আইডি দেখে লারাভেল নিজেই ডাটাবেজ থেকে ওই শেল্টার খুঁজে এনে দেয়। আমাকে নিজে কোয়্যারি লিখতে হয় না।"

🐞 **কীভাবে উত্তর দেবেন:** লাইন দেখালে প্রথমে বলুন এটা কোন মেথড (পড়তে `index`, বানাতে `store`, মুছতে `destroy`), তারপর ওই লাইনের কাজ এক বাক্যে বলুন।

২.২ — Model ফাইল (যেমন `Shelter.php`)

```
class Shelter extends Model // এটি একটি টেবিলের প্রতিনিধি ক্লাস
{
    use HasFactory; // টেস্ট ডাটা বানানোর সুবিধা

    protected $primaryKey = 'shelter_id'; // এই টেবিলের আইডি-কলাম কোনটা

    protected $fillable = [ // কোন কোন কলামে ডাটা লেখা যাবে
        'area_id', 'shelter_name', 'capacity', 'occupancy', 'status',
    ];

    public function affectedArea(): BelongsTo // সম্পর্ক: শেল্টার একটি এরিয়ার
    {
        return $this->belongsTo(AffectedArea::class, 'area_id', 'area_id');
    }
}
```

ট্র্যাপ প্রশ্ন ও উত্তর:

- "`$fillable` কেন দরকার?" → "নিরাপত্তার জন্য। লারাভেল শুধু `$fillable` -এ থাকা কলামগুলোতেই ডাটা লিখবে। এতে কেউ চাইলেই অন্য কোনো কলামে (যেমন আইডি) জোর করে ডাটা ঢোকাতে পারবে না। একে বলে **Mass Assignment Protection**।"

- "\$primaryKey কেন লিখতে হলো?" → "লারাভেল ডিফল্ট আইডি-কলামকে id ধরে নেয়। আমাদের টেবিলে নাম shelter_id, তাই জানিয়ে দিলাম।"
- "belongsTo মানে কী?" → "একটি শেল্টার একটি নির্দিষ্ট এলাকার অন্তর্গত। অর্থাৎ শেল্টার টেবিলে area_id ফরেন-কি আছে যা affected_areas টেবিলকে নির্দেশ করে।"
- "belongsTo -এর তিনটি আর্গুমেন্ট কী?" → "১) কোন মডেলের সাথে সম্পর্ক (AffectedArea), ২) এই টেবিলের ফরেন-কি (area_id), ৩) ওই টেবিলের প্রাইমারি কি (area_id)।"

 **মনে রাখুন:** মডেল = টেবিলের মানচিত্র। এটি বলে দেয় টেবিলের নাম, আইডি কোনটা, কোন কলামে লেখা যাবে, আর অন্য টেবিলের সাথে সম্পর্ক কী।

২.৩ — ফ্রন্টএন্ড ফাইল (operations-workspace.tsx)

এটি সবচেয়ে বড় ফাইল। স্যার এখান থেকে যেকোনো অংশ দেখাতে পারেন।

(ক) ফর্ম সাবমিট হ্যান্ডলার — handleAddShelter

```
const handleAddShelter = async (e: React.FormEvent) => {
  e.preventDefault(); // পেজ রিফ্রেশ আটকাই
  if (!shelterForm.shelter_name || !shelterForm.capacity) {
    toast.error("Please fill out shelter name and capacity."); // ফাঁকা হলে সতর্কতা
    return; // এখানেই থামি, পাঠাই না
  }

  const payload = { // কোন কোন ডাটা পাঠাব
    shelter_name: shelterForm.shelter_name,
    area_id: Number(shelterForm.area_id),
    capacity: Number(shelterForm.capacity),
    occupancy: Number(shelterForm.occupancy || 0),
    status: shelterForm.status,
  };

  if (!isOffline) { // ব্যাকএন্ড চালু থাকলে
    try {
      const res = await api.admin.shelters.create(payload); // POST পাঠাই
      setShelters([res.data, ...shelters]); // নতুনটা তালিকার সামনে বসাই
      toast.success(`Shelter saved to backend (ID: ${res.data.shelter_id})`);
    } catch (err) {
      toast.error("Failed to save shelter to backend."); // ব্যর্থ হলে বার্তা
    }
  } else { // ব্যাকএন্ড বন্ধ থাকলে (অফলাইন)
    // লোকালভাবে একটি আইডি বানিয়ে তালিকায় যোগ করি
  }

  setShelterForm({ ... }); // ফর্ম আবার খালি করে দিই
};
```

ট্র্যাপ প্রশ্ন ও উত্তর:

- `"e.preventDefault() কেন?"` → "ফর্ম সাবমিট করলে ব্রাউজার ডিফল্ট পেজ রিলোড করে। এটা আটকাই, যাতে ডাটা আগে পাঠাতে পারি।"
- `"Number(...)" কেন দিলে?"` → "ফর্মের ইনপুট ডিফল্ট স্ট্রিং হয়। ডাটাবেজে পাঠানোর আগে সেটাকে সংখ্যায় বদলাই, নাহলে টাইপ-ভুল হবে।"
- `"await কী?"` → "এটি একটি অ্যাসিনক্রোনাস অপারেশন। `await` দিয়ে অপেক্ষা করি যতক্ষণ না ব্যাকএন্ড থেকে উত্তর আসে।"
- `"অফলাইন মোড কেন রেখেছ?"` → "যাতে ব্যাকএন্ড বন্ধ থাকলেও ইউজার ইন্টারফেস কাজ করে দেখানো যায়। এটি একটি ফলব্যাক (fallback) ব্যবস্থা।"
- `"[res.data, ...shelters]" কী করছে?"` → "নতুন শেল্টারকে তালিকার শুরুতে বসাবি, বাকিগুলো পেছনে রাখছি। `...` হলো স্প্রেড অপারেটর।"

(খ) টেবিলে ডাটা দেখানো (`.map()` অংশ)

```
{shelters.map((s) => (                                // প্রতিটি শেল্টারের জন্য একটি সারি
  <tr key={s.shelter_id}>                                // ইউনিক আইডি দিলাম (রিঅ্যাক্টের নিয়ম)
    <td>#{s.shelter_id}</td>
    <td>{s.shelter_name}</td>
    <td>{s.capacity}</td>
    ...
  </tr>
))}
```

- `"key={s.shelter_id}" কেন?"` → "রিঅ্যাক্ট প্রতিটি সারিকে আলাদা করে চেনার জন্য একটি ইউনিক আইডি চায়। না দিলে ডাটা বদলালে ভুল জায়গায় আপডেট হতে পারে।"

 **কীভাবে উত্তর দেবেন:** ফ্রন্টএন্ডে লাইন দেখালে বলুন — এটা কি (১) ফর্ম যাচাই, (২) ডাটা পাঠানো, (৩) ডাটা দেখানো, নাকি (৪) ফর্ম রিসেট। তারপর এক বাক্যে কাজ বলুন।

৩. ডাটাবেজ JOIN ও লজিক

 এটিই সবচেয়ে বেশি প্রশ্ন আসা এলাকা। ভালো করে পড়ুন।

৩.১ — মূল রিপোর্ট ক্যোয়ারি (`admin_shelter_capacity_report.sql`)

```
SELECT
    aa.area_id,
    aa.severity AS area_severity,
    aa.affected_population,
    COUNT(s.shelter_id) AS total_shelters,
    COALESCE(SUM(s.capacity), 0) AS total_capacity,
    COALESCE(SUM(s.capacity - s.occupancy), 0) AS total_capacity_available
FROM affected_areas aa
LEFT JOIN shelters s ON s.area_id = aa.area_id
GROUP BY aa.area_id, aa.severity, aa.affected_population
ORDER BY total_capacity_available DESC;
```

৩.২ — কোন কোন টেবিল জয়েন হলো এবং কেন?

টেবিল	ভূমিকা	জয়েনের কারণ
<code>affected_areas</code> (aa)	মূল টেবিল	প্রতিটি এলাকার তথ্য দেখাতে হবে
<code>shelters</code> (s)	যুক্ত টেবিল	ওই এলাকায় কয়টি শেল্টার ও তাদের ধারণক্ষমতা দেখাতে

জয়েনের শর্ত: `ON s.area_id = aa.area_id` → শেল্টার টেবিলের `area_id` আর এলাকা টেবিলের `area_id` মিলিয়ে দেখা হচ্ছে। অর্থাৎ কোন শেল্টার কোন এলাকার।

🔗 **এক বাক্যে:** "স্যার, `affected_areas` -এর সাথে `shelters` জয়েন করেছি `area_id` দিয়ে, যাতে প্রতিটি এলাকার জন্য সেখানকার শেল্টারগুলোর মোট ধারণক্ষমতা বের করতে পারি।"

৩.৩ — `LEFT JOIN` কেন? `INNER JOIN` কেন নয়?

এটি সবচেয়ে সম্ভাব্য ট্র্যাপ প্রশ্ন।

- `LEFT JOIN` → বাম টেবিলের (এলাকা) **সব** রেকর্ড আসবে, এমনকি যদি তার কোনো শেল্টার না-ও থাকে। তখন শেল্টার-কলামগুলো `NULL` হবে।
- `INNER JOIN` → শুধু সেই এলাকাগুলো আসবে যাদের **অন্তত একটি** শেল্টার আছে। যেসব এলাকায় শেল্টার নেই, তারা হারিয়ে যাবে।

আমরা `LEFT JOIN` বেছে নিয়েছি কারণ: ত্রাণ ব্যবস্থাপনায় জানা জরুরি **কোন এলাকায় এখনো কোনো শেল্টারই নেই** — সেখানে জরুরি ব্যবস্থা নিতে হবে। `INNER JOIN` দিলে এই গুরুত্বপূর্ণ তথ্য লুকিয়ে যেত।

🔗 **উত্তর:** "`LEFT JOIN` দিয়েছি যাতে শেল্টার-বিহীন এলাকাগুলোও রিপোর্টে দেখা যায় এবং তাদের ধারণক্ষমতা `0` দেখায়। এতে প্রশাসন বুঝতে পারে কোথায় শেল্টার বসাতে হবে।"

৩.৪ — এখানে কি কোনো Triple JOIN আছে?

সরাসরি উত্তর: না। এই রিপোর্টে মাত্র দুইটি টেবিল জয়েন হয়েছে (`affected_areas` + `shelters`)। এটি একটি দ্বি-টেবিল (2-table) জয়েন।

তবে আপনি চাইলে ট্রিপল জয়েন সম্ভব দেখাতে পারেন, কারণ `affected_areas` টেবিলে `disaster_id` ফরেন-কি আছে:

```
-- উদাহরণ: তিনটি টেবিল জয়েন (শেল্টার ← এলাকা ← দুর্যোগ)
SELECT aa.area_id, d.disaster_name, COUNT(s.shelter_id) AS shelter_count
FROM affected_areas aa
INNER JOIN disasters d ON d.disaster_id = aa.disaster_id
LEFT JOIN shelters s ON s.area_id = aa.area_id
GROUP BY aa.area_id, d.disaster_name;
```

 **কী বলবেন:** "স্যার, আমার মূল রিপোর্টে দুইটি টেবিলের জয়েন আছে। তবে `affected_areas` -এর সাথে `disasters` -ও যুক্ত থাকায় চাইলে তিনটি টেবিলের জয়েনও করা সম্ভব — যেটা এলাকা এবং সংশ্লিষ্ট দুর্যোগ দুটোর তথ্যই দেখাবে।"

৩.৫ — WHERE ক্লজ কোথায় ও কেন?

দুটি ভিন্ন ক্যোয়ারিতে WHERE দুইভাবে ব্যবহৃত:

(ক) সাবক্যোয়ারিতে WHERE (Critical এলাকার শেল্টার):

```
SELECT shelter_name, capacity FROM shelters
WHERE area_id IN (SELECT area_id FROM affected_areas WHERE severity = 'Critical')
ORDER BY capacity DESC;
```

- এখানে WHERE ফিল্টার করছে: শুধু সেই শেল্টার দেখাও যাদের এলাকা Critical।
- ভেতরের SELECT আগে চলে Critical এলাকাগুলোর আইডি বের করে, তারপর বাইরের WHERE ... IN সেই আইডিগুলোর শেল্টার দেখায়।

(খ) মূল রিপোর্টে WHERE নেই — কিন্তু আছে GROUP BY ও ORDER BY :

- মূল রিপোর্টে আমরা কোনো সারি বাদ দিচ্ছি না (সব এলাকা দরকার), তাই WHERE নেই।
- বরং GROUP BY দিয়ে প্রতিটি এলাকার জন্য আলাদা হিসাব করছি।

গুরুত্বপূর্ণ পার্থক্য — WHERE বনাম HAVING :

- WHERE → জয়েনের পরে, কিন্তু আগ্রিগেশনের আগে সারি ফিল্টার করে।
- HAVING → GROUP BY ও আগ্রিগেশনের পরে ফিল্টার করে।
- যেমন: "যে এলাকায় শেল্টার-সংখ্যা ২-এর বেশি" দেখাতে চাইলে HAVING COUNT(s.shelter_id) > 2 লাগবে, WHERE দিয়ে এটা করা যায় না।

🔗 **উত্তর:** "WHERE ব্যবহার করি সারি ফিল্টার করতে, আর জয়েনের শর্ত ON দিয়ে টেবিল মেলাই। মূল রিপোর্টে কোনো সারি বাদ দেওয়ার দরকার নেই বলে WHERE নেই — সব এলাকা দেখাতে চাই।"

৩.৬ — অ্যাগ্রিগেট ফাংশনগুলোর কাজ

ফাংশন	কী করে
COUNT(s.shelter_id)	ওই এলাকায় কয়টি শেল্টার (ফাঁকা/ NULL গোনো না)
SUM(s.capacity)	সব শেল্টারের মোট ধারণক্ষমতা
SUM(s.capacity - s.occupancy)	মোট খালি জায়গা (ধারণক্ষমতা - যারা এখন আছে)
COALESCE(x, 0)	যদি ফল NULL হয়, তবে 0 দেখাও
GROUP BY aa.area_id	প্রতিটি এলাকার জন্য আলাদা হিসাব করো

COALESCE কেন দরকার? যে এলাকায় কোনো শেল্টার নেই, সেখানে SUM ফলাফল NULL হয়। COALESCE(..., 0) সেটাকে 0 -এ বদলায়, যাতে রিপোর্টে 0 দেখায়, ফাঁকা বা NULL নয়।

🔗 **উত্তর:** "COALESCE দিয়েছি কারণ LEFT JOIN -এ শেল্টার-বিহীন এলাকার যোগফল NULL আসে। আমি চাই ওখানে স্পষ্ট 0 দেখাক।"

৪. কলাম, গারবেজ ডাটা ও অমিল

৪.১ — জয়েনের পর অতিরিক্ত/অপ্রত্যাশিত কলাম কেন আসে?

যদি আপনি জয়েনে SELECT * লেখেন, তাহলে দুই টেবিলের সব কলাম চলে আসে। সমস্যা হয় তখন যখন দুই টেবিলে একই নামের কলাম থাকে:

- affected_areas -এ আছে: area_id, created_at, updated_at
- shelters -এও আছে: area_id, created_at, updated_at

তাই SELECT * করলে area_id ও created_at দুইবার আসে — কোনটা কোন টেবিলের বোঝা যায় না। এগুলোই "অতিরিক্ত/অপ্রত্যাশিত কলাম"।

সমাধান (আমরা যা করেছি): SELECT * না লিখে শুধু দরকারি কলাম লিখেছি এবং উপনাম (alias) দিয়েছি:

```
aa.area_id, aa.severity AS area_severity
```

aa. আর s. দিয়ে স্পষ্ট করেছি কোন কলাম কোন টেবিলের।

🔗 **উত্তর:** "স্যার, SELECT * করলে দুই টেবিলের সব কলাম আসে, আর একই নামের কলাম (যেমন area_id, created_at) ডুপ্লিকেট হয়ে যায়। তাই আমি নির্দিষ্ট কলাম লিখেছি এবং টেবিল-উপনাম দিয়ে স্পষ্ট

করেছি।"

৪.২ — গারবেজ/অপ্রত্যাশিত ডাটা কোথা থেকে আসে?

কয়েকটি উৎস:

1. **NULL ফরেন-কি:** `shelters.area_id` ফাঁকা রাখা যায়। ফলে এমন শেল্টার থাকতে পারে যা কোনো এলাকার সাথে যুক্ত নয়। জয়েনে এরা হয়তো দেখাবে না, কিন্তু `NULL` মান হিসেবে ডাটাবেজে থাকে।
2. **ON DELETE SET NULL আচরণ:** আমরা ডিজাইন করেছি — কোনো এলাকা মুছে তার শেল্টার মুছে না গিয়ে শুধু `area_id` ফাঁকা (`NULL`) হয়ে যায়। এটি ইচ্ছাকৃত, যাতে শেল্টারের ইতিহাস হারিয়ে না যায়।
3. **ফ্রি-টেক্সট কলাম:** `shelter_name`, `status` -এর মতো কলামে ইউজার যা খুশি লিখতে পারে, তাই ভুল বানান বা অসঙ্গত মান ঢুকতে পারে।
4. **সীমাহীন স্ট্যাটাস:** `status` একটি সাধারণ `VARCHAR`, তাই কেউ `open / Open / OPEN` লিখলে তিন রকম মান তৈরি হতে পারে।

🔗 **উত্তর:** "গারবেজ ডাটা মূলত আসে ফাঁকা ফরেন-কি, ইচ্ছাকৃত `SET NULL` আচরণ, আর ফ্রি-টেক্সট কলাম থেকে। আমরা ভ্যালিডেশন দিয়ে (`required`, `max`, `integer`) ইনপুট-সাইডে এটা নিয়ন্ত্রণ করি।"

৪.৩ — অ্যাগ্রিগেট/রো-কাউন্ট কেন ডাটাবেজ ও ফ্রন্টএন্ডে আলাদা দেখায়?

এটি একটি দুর্দান্ত "ফাঁদ" প্রশ্ন। কারণগুলো:

(ক) ডাটাবেজে `GROUP BY` কিন্তু ফ্রন্টএন্ডে ফিল্টার:

- ডাটাবেজ কোয়ারি প্রতিটি **এলাকা**-ভিত্তিক হিসাব করে (`GROUP BY`)।
- ফ্রন্টএন্ড কিন্তু শুধু **শেল্টার-তালিকা** দেখায়, প্রতিটি এলাকার যোগফল নয়।
- তাই ডাটাবেজ রিপোর্টে দেখা "মোট ৩টি শেল্টার" আর ফ্রন্টএন্ডের তালিকায় "১০টি শেল্টার" — দুটো **ভিন্ন জিনিস**।

(খ) `LEFT JOIN` -এর `NULL` রো:

- রিপোর্টে শেল্টার-বিহীন এলাকাও `0` নিয়ে উপস্থিত থাকে।
- কিন্তু শুধু `shelters` টেবিল দেখালে ওই এলাকাগুলো একেবারেই থাকে না। ফলে "রো-সংখ্যা" আলাদা লাগে।

(গ) `COUNT(shelter_id)` বনাম `COUNT(*)`:

- `COUNT(s.shelter_id) → NULL` আইডি গোনো না।
- `COUNT(*) → সব রো গোনো, NULL সহ।`
- তাই শেল্টার-বিহীন এলাকায় `COUNT(s.shelter_id)` দেয় `0`, কিন্তু `COUNT(*)` দিত `1` (একটি ম্যাচিং-না-পাওয়া রো)।

(ঘ) ডাটা বদলে যাওয়া (টাইমিং):

- ডাটাবেজে ক্যোয়ারি চালানোর সময় আর ফ্রন্টএন্ড ডাটা আনার সময়ের মাঝে নতুন রেকর্ড যোগ/মুছা হলে সংখ্যা বদলাবে।

(৩) ফ্রন্টএন্ড শুধু লোড-হওয়া ডাটা দেখায়:

- ফ্রন্টএন্ড যা এপিআই থেকে পেয়েছে শুধু সেটাই দেখায়। পেজিনেশন বা ফিল্টার থাকলে পুরো ডাটাবেজের সংখ্যা দেখাবে না।

উত্তর: "স্যার, অমিলটা আসলে ভুল নয় — দুটো ভিন্ন হিসাব। ডাটাবেজ ক্যোয়ারি `GROUP BY` দিয়ে প্রতি এলাকার যোগফল বের করে, আর ফ্রন্টএন্ড শুধু শেল্টারের কাঁচা তালিকা দেখায়। তাছাড়া `COUNT(shelter_id)` ফাঁকা মান গোনে না, কিন্তু `COUNT(*)` গোনে — এই পার্থক্যই সংখ্যা আলাদা দেখায়।"

৫. র‍্যাপিড-ফায়ার প্রশ্নোত্তর

প্রশ্ন	দ্রুত উত্তর
মডেল আর কন্ট্রোলারের তফাত?	মডেল = টেবিলের মানচিত্র; কন্ট্রোলার = রিকোয়েস্ট হ্যান্ডেল করার লজিক।
<code>fillable</code> না দিলে কী হতো?	নিরাপত্তা ঝুঁকি — যেকোনো কলামে ডাটা ঢোকানো যেত।
<code>LEFT JOIN</code> আর <code>INNER JOIN</code> ?	<code>LEFT JOIN</code> বাম টেবিলের সব রাখে, <code>INNER JOIN</code> শুধু মিলে যাওয়া রাখে।
সাবক্যোয়ারি কেন, জয়েন কেন নয়?	উভয়েই কাজ হয়; সাবক্যোয়ারি এখানে পরিস্কার কারণ "আগে আইডি বের করো, তারপর শেল্টার দেখাও"।
<code>GROUP BY</code> না দিলে কী হতো?	সব এলাকা মিলিয়ে একটিমাত্র যোগফল আসত, প্রতি-এলাকা হিসাব হতো না।
<code>COALESCE</code> ছাড়া কী হতো?	শেল্টার-বিহীন এলাকায় যোগফল ০-এর বদলে <code>NULL</code> দেখাত।
ফ্রন্টএন্ড থেকে ডাটা কীভাবে যায়?	<code>fetch</code> দিয়ে <code>POST</code> রিকোয়েস্ট, <code>payload</code> JSON আকারে ব্যাকএন্ডে পাঠাই।
201 , 204 কী?	201 = তৈরি হয়েছে; 204 = মুছে ফেলা হয়েছে, কোনো বডি নেই।
অফলাইন মোড কেন?	ব্যাকএন্ড বন্ধ থাকলেও ডেমো চালানোর ফলব্যাক ব্যবস্থা।
ডাটাবেজে ৩টি টেবিলই কি জয়েন হয়?	মূল রিপোর্টে দুইটি; চাইলে <code>disasters</code> যোগ করে তিনটি করা যায়।

৬. রেফারেন্স টেবিল

আপনার তিন টেবিল

টেবিল	কলাম	টাইপ	মানে
shelters	shelter_id	INT (PK)	শেল্টারের আইডি
	area_id	INT (FK, NULL)	কোন এলাকার (ফাঁকা হতে পারে)
	shelter_name	VARCHAR(150)	শেল্টারের নাম
	capacity	INT	মোট ধারণক্ষমতা
	occupancy	INT	বর্তমানে কতজন আছে
	status	VARCHAR(50)	open / full / closed
warehouses	warehouse_id	INT (PK)	গুদামের আইডি
	warehouse_name	VARCHAR(150)	গুদামের নাম
	location_id	INT (NULL)	লোকেশন কোড
	manager_id	INT (FK, NULL)	দায়িত্বপ্রাপ্ত ব্যবহারকারী
donations	donation_id	BIGINT (PK)	দানের আইডি
	donation_kind	VARCHAR(50)	কী ধরনের দান
	amount	DECIMAL(12,2)	পরিমাণ/মূল্য
	status	VARCHAR(50)	received / stored / distributed

ফরেন-কি সম্পর্ক

ফরেন-কি	কোথা থেকে	কোথায় যায়	আচরণ
fk_shelters_area	shelters.area_id	affected_areas.area_id	ON DELETE SET NULL
fk_warehouses_manager	warehouses.manager_id	users.user_id	ON DELETE SET NULL

🔗 **শেষ বাক্য:** "স্যার, আমি তিনটি মডিউল পুরো স্ট্যাকে তৈরি করেছি, ডাটাবেজে সঠিক ফরেন-কি ও ভ্যালিডেশন রেখেছি, এবং `LEFT JOIN + GROUP BY + COALESCE` দিয়ে একটি বাস্তবসম্মত ক্যাপাসিটি-রিপোর্ট বানিয়েছি যা শেল্টার-বিহীন এলাকাও চিহ্নিত করে।"

শুভকামনা! আপনি প্রস্তুত। 🎓